## Lists

- Lists play a fundamental role in simulation, serving as ordered collections of records representing entities or event notices.
- Each list has a head pointing to the first item, allowing traversal to subsequent items, and possibly a tail pointing to the last item.
- Records on the list typically include fields for data and pointers to the next (and sometimes previous) record.
- List processing involves common operations like adding or removing records from the top or bottom of the list, or at specific positions based on ranking rules. Efficient list processing is crucial for simulation performance.
- In event-scheduling simulations, when advancing time, the imminent event is removed from the top of the Future Event List (FEL) first. Other removals or additions may occur based on specific conditions or ranking rules.
- In computer simulation, records can be stored in arrays or dynamically allocated structures/classes tracked by pointers.
- Most simulation packages use dynamically allocated records and pointers for list management, though arrays provide a simpler conceptual explanation before delving into linked lists.

## Arrays for list processing

- Arrays are a straightforward method for storing lists in simulation, where each array element corresponds to a record.
- However, arrays have limitations, such as fixed size and inefficiency when adding or removing items from the middle of a list.
- In array-based list storage, two methods for maintaining list order are commonly used:
    - The first method physically rearranges records in memory, which becomes inefficient for larger lists.

    - The second method, more commonly used, employs a head pointer indicating the top of the list and next pointers in each record to track the logical ordering of items without physically moving records.
- In the latter method, each record includes a next pointer, denoted as *R(i, next)*, storing the index or pointer of the next record in the list. Additionally, a head pointer, such as *headptr*, points to the record at the top of the list. This approach efficiently manages list order without the need for physical rearrangement of records.

**Example: A list for dump trucks at weighing que**

- In this example we implement the weighing que for the dump truck problem as an array.
- Each record represents a dump truck, with fields for the dump truck's identifier, arrival time at the weigh queue, and a pointer to the next record in the queue.

[DTi, arrival time at weigh queue, next index]

- When a waiting line of dump trucks occurs at the weigh queue, the list of entities in the queue is updated accordingly.
- At time 0, the records are initiated as follows:

R(1) = [DT1, 0.0, 0]

R(2) = [DT2, 0.0, 0]

.

.

R(6) = [DT6, 0.0, 0]

- At time 10 in the dump truck problem, we have a weighing que of DT3, DT2, DT4. Therefore the array is updated as follows:

*headptr = 3*

R(1) = [DT1, 0.0, 0]

R(2) = [DT2, 10.0, 4]

R(3) = [DT3, 5.0, 2]

R(4) = [DT4, 10.0, 0]

R(5) = [DT5, 0.0, 0]

R(6) = [DT6, 0.0, 0]

*tailptr = 4*

- Traversing the queue involves following the next pointers from the head to the tail, or vice versa. Adding or removing dump trucks from the queue is straightforward, involving simple updates to the head and tail pointers.

**Linked lists for list processing**

- In languages like C++ and Java, as well as in simulation languages, entity records and event-notice records are dynamically allocated when needed.
- The language or operating system manages a linked list of free memory chunks and allocates memory upon request.
- When an entity exits the system or an event notice is no longer needed, the corresponding records are freed, making the memory available for reuse.
- Dynamic allocation involves referencing records using pointers instead of array indices.
- When a record is allocated, a pointer to the allocated record is returned, which must be stored for later use. Pointers represent the physical or logical address in memory where the record is stored.

- The notation for records remains the same as in the previous section: Entities are represented as [ID, attributes, next pointer], while event notices are represented as [event type, event time, other data, next pointer]. However, we no longer use array notation like R(i) because records in a linked list are not stored contiguously in memory.
- To access a specific record in a linked list, traversal is required, counting items until the desired record is reached, as there's no direct way to retrieve the i-th record as with arrays.